

## 格式化文档

默认视图为阅读优化。如有需要，可在下方查看源 Markdown。

# 系统架构文档

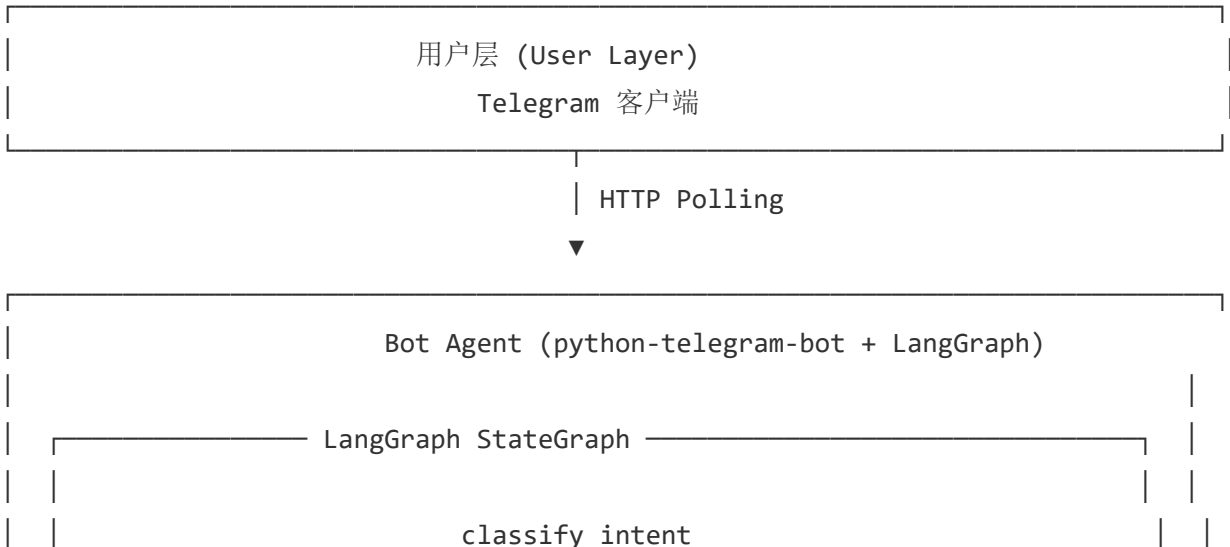
本文档详细描述 Telegram 智能助手后台的技术架构、组件设计、数据流和部署方案。

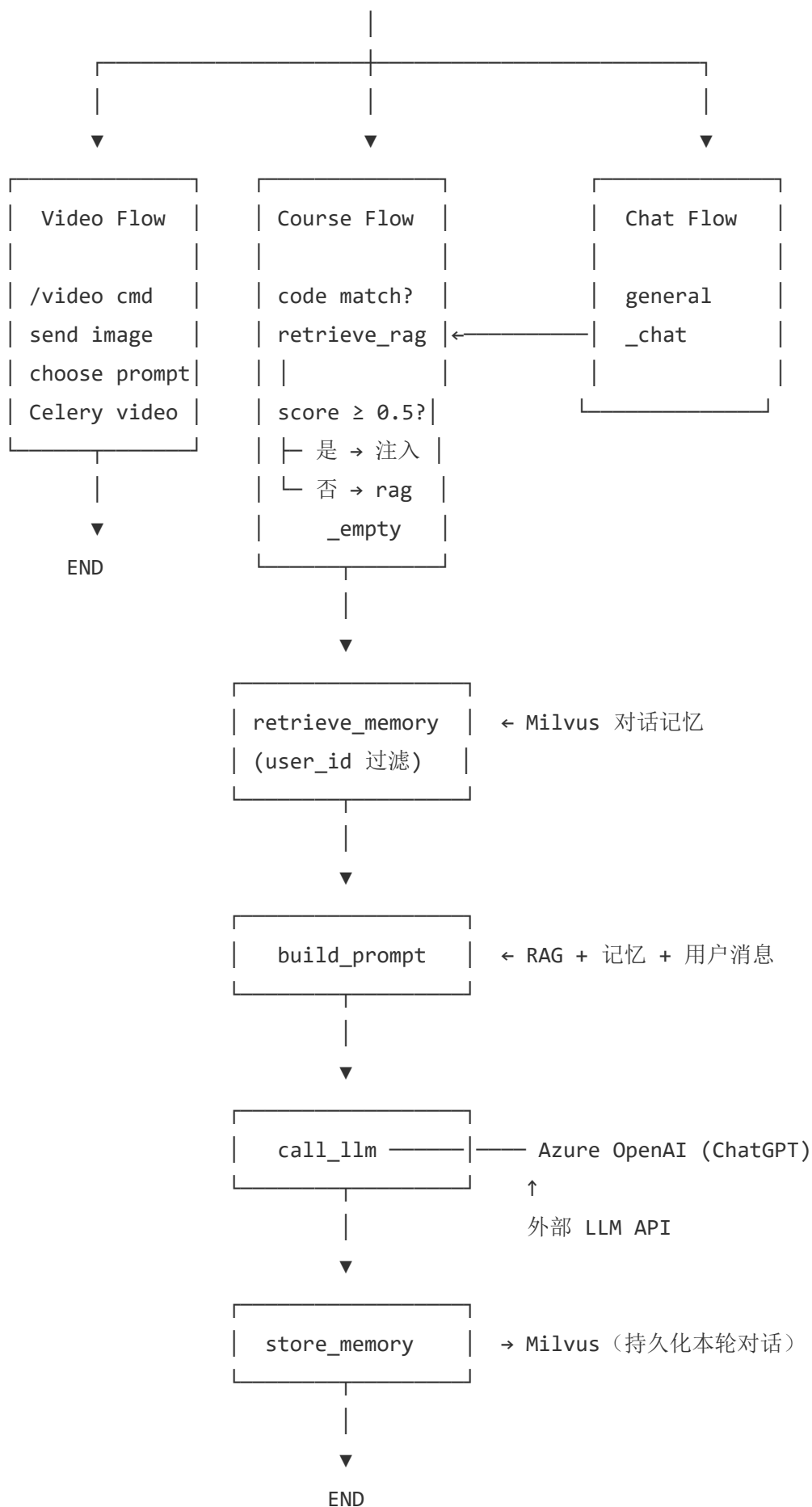
## 目录

- 1. 架构总览
- 2. 核心组件
- 3. LangGraph 状态机
- 4. RAG 检索系统
- 5. 数据流分析
- 6. 容器化部署
- 7. 配置体系
- 8. 扩展性设计
- 9. 高可用设计

## 1. 架构总览

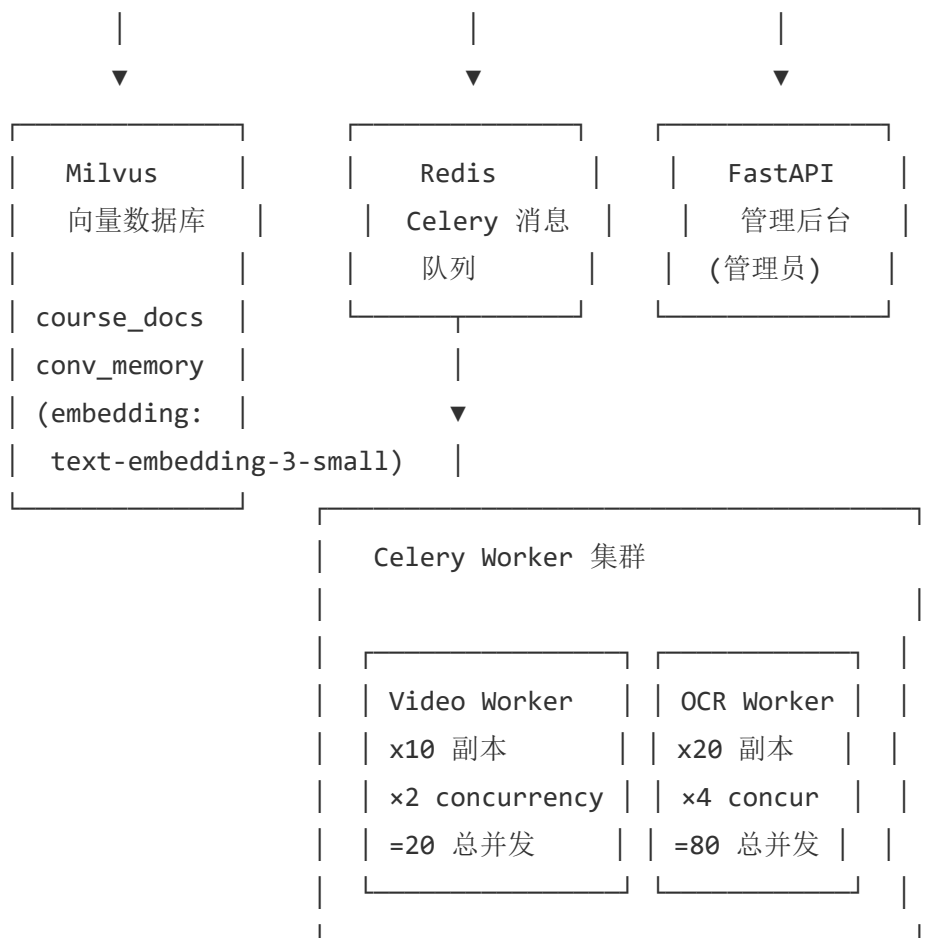
### 整体架构图





另: `analyze_document` → Celery OCR + 异步入库 Milvus → END

`receive_video_image/prompt` → Celery 视频生成 → END



## 核心设计原则

| 原则        | 说明                                                                                                               |
|-----------|------------------------------------------------------------------------------------------------------------------|
| 统一 RAG 入口 | 无论课程代码还是自然语言查询，均先经过 Milvus Hybrid RAG 语义检索                                                                       |
| 异步非阻塞     | 全链路异步（ <code>async/await</code> ），单进程即可处理大量并发对话                                                                  |
| 混合检索      | 课程查询：BM25 稀疏匹配 + Milvus 稠密向量，RRF 融合双引擎                                                                           |
| 对话记忆      | 每次对话自动存入 Milvus <code>conversation_memory</code> 集合，按 <code>user_id</code> 隔离；下次提问时语义检索历史并注入 <code>prompt</code> |
| 全量 Milvus | 课程数据 + 对话记忆 + 用户 PDF 全部托管在 Milvus，无外部数据库依赖                                                                       |
| 阈值安全      | RAG 结果低于相似度阈值（0.5）时，LLM 如实回复"未找到"而非编造                                                                            |
| 松耦合       | 各组件通过消息队列/API 通信，可独立扩缩容                                                                                          |

## 2. 核心组件

### 2.1 Bot Agent

文件: chatbot\_agent.py

Telegram 机器人主程序，负责：

- 通过 python-telegram-bot 接收用户消息
- 构建初始 AgentState ，调用 langgraph\_app.ainvoke(state)
- 将 final\_response 返回给用户
- 同步 user\_data 状态标记（用于视频多轮对话）

消息路由策略：

| 消息类型      | 处理器                  | 说明                              |
|-----------|----------------------|---------------------------------|
| 文本消息      | handle_text          | 构建 AgentState ，调用 LangGraph 工作流 |
| /video 命令 | handle_video_command | 设置视频工作流标记                       |
| 图片/文档附件   | handle_attachment    | 根据 user_data 标记路由到视频或文档分析       |

### 2.2 LangGraph 工作流

文件: graph/workflow.py

StateGraph 节点注册表：

| 节点                   | 函数                        | 类型    | 说明                      |
|----------------------|---------------------------|-------|-------------------------|
| classify_intent      | classify_intent_node      | sync  | 意图分类（检查 user_data + 正则） |
| video_command        | video_command_node        | sync  | 初始化视频工作流                |
| receive_video_image  | receive_video_image_node  | async | 接收图片 → Celery 分析        |
| receive_video_prompt | receive_video_prompt_node | async | 接收 prompt → Celery 生成   |

| retrieve\_rag | retrieve\_rag\_node | async | Milvus 语义检索 + 阈值过滤 || retrieve\_memory |  
retrieve\_conversation\_memory\_node | async | Milvus 对话记忆检索（按 user\_id 隔离） ||

build\_prompt | build\_prompt\_node | sync | 组装 LLM 提示词（含对话记忆） || call\_llm |  
call\_llm\_node | async | ChatGPT API 调用 |  
| store\_memory | store\_conversation\_memory\_node | async | 对话存入 Milvus 记忆向量库 ||  
analyze\_document | analyze\_document\_node | async | PDF 文档分析 (Celery) |

## 2.3 AgentState 状态定义

文件: graph/state.py

```
class AgentState(TypedDict):  
    user_id: int # Telegram 用户 ID  
    user_message: str # 用户原始消息  
    intent: Optional[str] # 意图分类结果  
    course_code: Optional[str] # 课程代码  
    rag_context: str # Milvus RAG 检索结果  
    rag_empty: bool # RAG 是否无结果  
    conversation_memory_context: str # Milvus 对话记忆检索结果（跨会话持久化）  
    augmented_prompt: str # 组装后的 LLM 提示词  
    final_response: Optional[str] # LLM 回复  
    error: Optional[str] # 错误信息  
    waiting_for_video_image: bool # 等待视频图片  
    waiting_for_video_prompt: bool # 等待视频 prompt  
    video_image_base64: Optional[str] # 图片 base64  
    suggested_prompts: list[str] # AI 建议的动画提示  
    video_task_id: Optional[str] # Celery 任务 ID  
    celery_result: Optional[dict] # Celery 结果  
    _raw_update: Any # Telegram Update 对象  
    _raw_context: Any # Telegram Context 对象
```

## 2.4 ChatGPT 客户端

文件: ChatGPT\_HKBU.py

兼容双配置源（ConfigParser 和 Pydantic Settings）的 Azure OpenAI HTTP 客户端。

生产级特性：

- **指数退避重试**（3 次）：对 HTTP 429（限流）、5xx（服务端错误）和网络超时自动重试，间隔 2s→4s→8s

- **熔断器 (Circuit Breaker)**：连续 5 次失败后切断请求 30 秒，半开后试探恢复，防止雪崩
- **连接池优化**： `httpx.Limits(max_connections=200, max_keepalive_connections=50)` ，支持 200 并发请求
- **超时控制**：总体 60s，连接超时 10s

| 方法                                    | 类型                 | 说明             |
|---------------------------------------|--------------------|----------------|
| <code>submit()</code>                 | <code>async</code> | 异步文本补全（含重试+熔断） |
| <code>submit_sync()</code>            | <code>sync</code>  | 同步文本补全（含重试+熔断） |
| <code>submit_with_image()</code>      | <code>async</code> | 异步图片分析         |
| <code>submit_with_image_sync()</code> | <code>sync</code>  | 同步图片分析         |

## 2.5 FastAPI 管理界面

文件: `api_server.py` 、 `api_templates/index.html`

课程数据管理服务，程序员通过浏览器上传 CSV/JSON 数据到 Milvus。

| 端点                       | 方法                | 说明             |
|--------------------------|-------------------|----------------|
| <code>/admin</code>      | <code>GET</code>  | Web 管理后台       |
| <code>/api/upload</code> | <code>POST</code> | 上传 CSV/JSON 文件 |
| <code>/api/ingest</code> | <code>POST</code> | 直接注入 JSON 数据   |
| <code>/api/health</code> | <code>GET</code>  | 健康检查           |
| <code>/api/stats</code>  | <code>GET</code>  | 查看 Milvus 数据统计 |

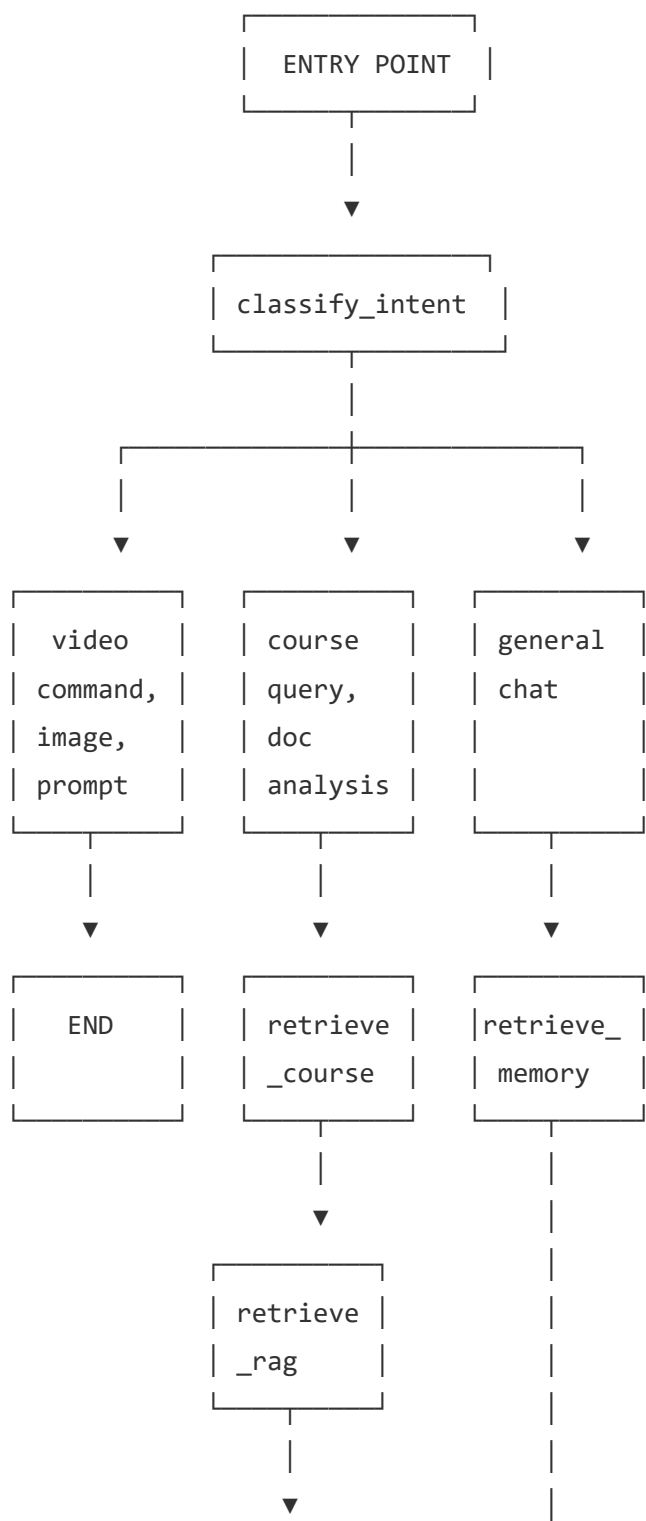
## 2.6 辅助模块

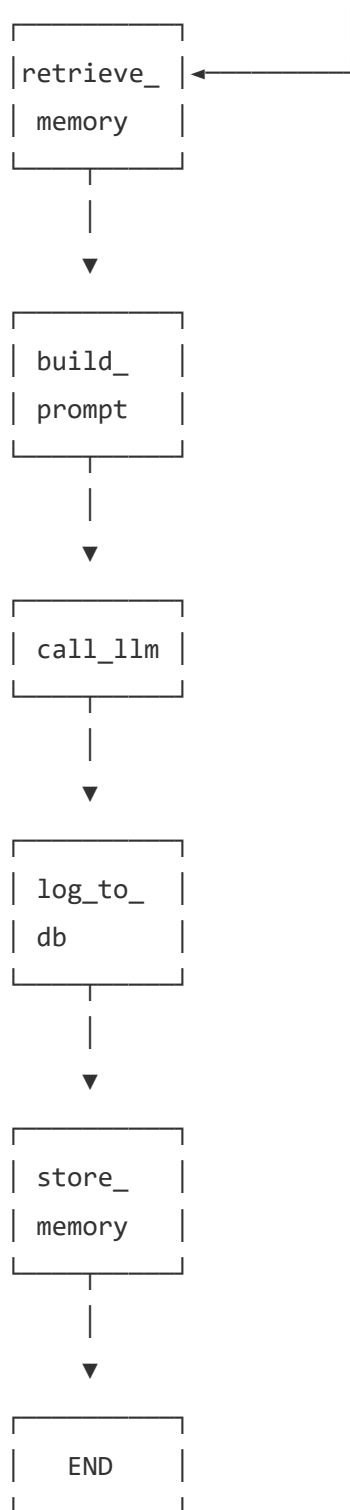
| 模块             | 文件                               | 说明                                              |
|----------------|----------------------------------|-------------------------------------------------|
| Settings       | <code>configs/settings.py</code> | Pydantic 配置管理，支持 <code>config.ini</code> + 环境变量 |
| Image-to-Video | <code>app/video.py</code>        | SiliconFlow Wan-AI API 封装                       |
| Celery Tasks   | <code>workers/tasks.py</code>    | 视频生成、文档分析、图片分析任务                                |

| 模块           | 文件               | 说明                 |
|--------------|------------------|--------------------|
| Worker Entry | workers/entry.py | Celery Worker 启动入口 |

### 3. LangGraph 状态机

#### 3.1 workflow 拓扑





## 3.2 条件路由逻辑

`intent_router` 函数根据 `classify_intent_node` 输出的 `intent` 字段决定下一节点：

```
{  
    "video_command":          "video_command",          # → 设置标记 → END  
    "receive_video_image":    "receive_video_image",    # → 分析图片 → END  
    "receive_video_prompt":   "receive_video_prompt",    # → 生成视频 → END
```



|                     |                     |                                |
|---------------------|---------------------|--------------------------------|
| "retrieve_course":  | "retrieve_course",  | # → DB 查询 → RAG → memory → ... |
| "analyze_document": | "analyze_document", | # → PDF 分析 → END               |
| "general_chat":     | "retrieve_rag",     | # → RAG 检索 → 记忆 → LLM → 存入记    |
| }                   |                     |                                |

### 3.3 意图分类策略

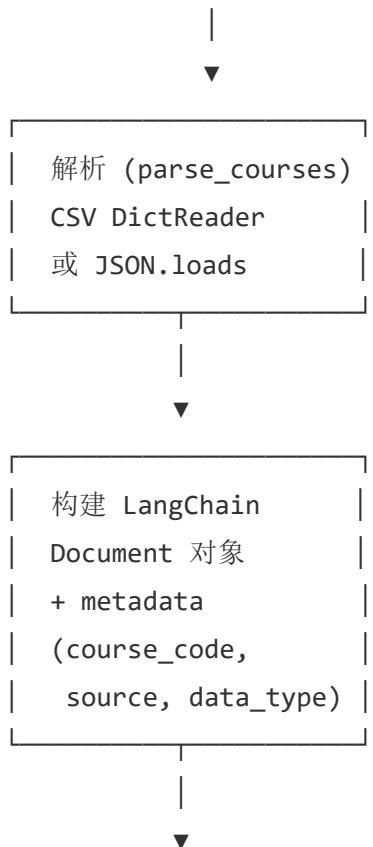
classify\_intent\_node 按优先级依次检查：

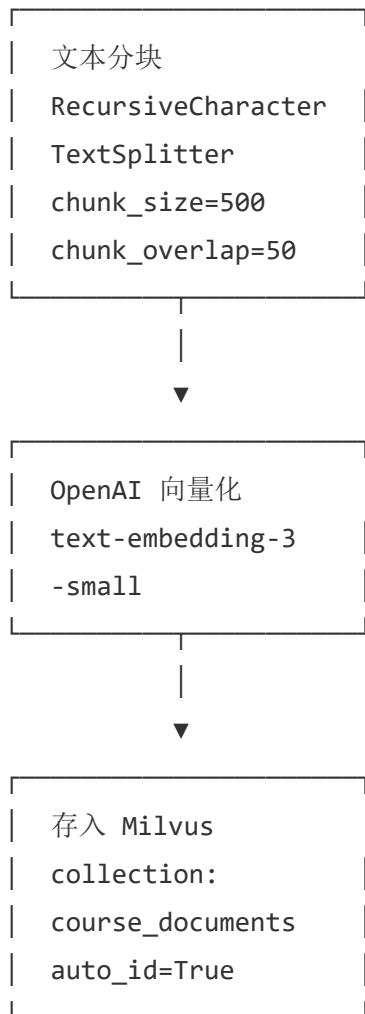
1. user\_data.get("waiting\_for\_video\_prompt") → receive\_video\_prompt
2. user\_data.get("waiting\_for\_video\_image") → receive\_video\_image
3. msg.startswith("/video") → video\_command
4. re.search(r"[A-Z]{4}\d{4}", msg) → retrieve\_course (课程代码)
5. 默认 → general\_chat (含纯语义 RAG 检索课程名称)

## 4. RAG 检索系统

### 4.1 数据灌入流程

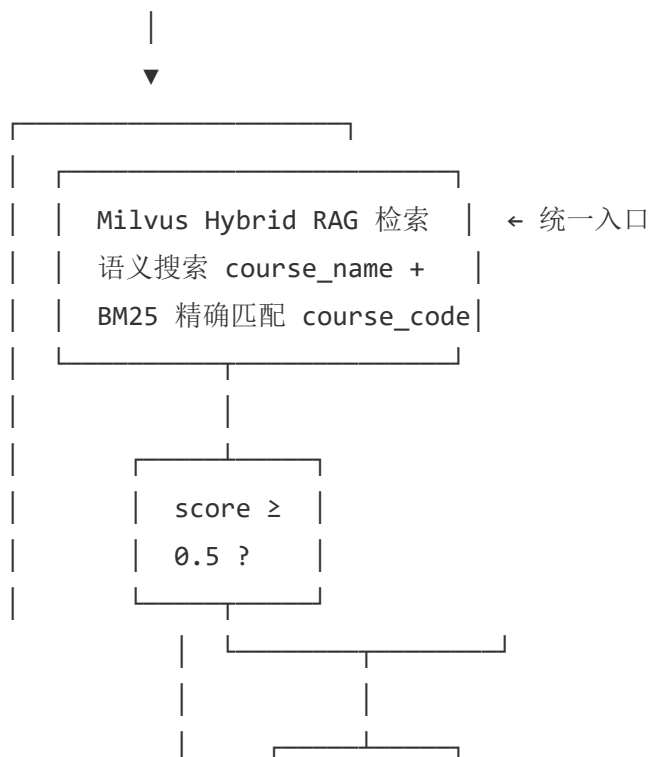
程序员准备 CSV/JSON → 上传到 FastAPI /api/upload

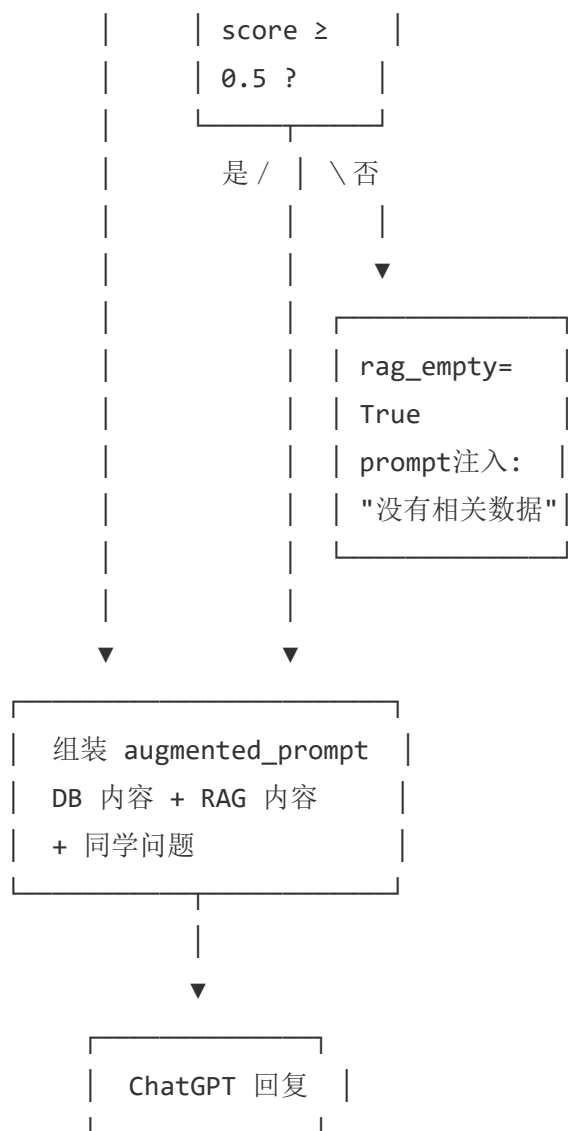




## 4.2 检索流程

用户问: "COMP7940 有什么作业?" / "Cloud Computing 什么时候上课?"





4.3 向量存储配置

| 配置项   | 值                      | 说明                 |
|-------|------------------------|--------------------|
| 嵌入模型  | text-embedding-3-small | OpenAI 第三代嵌入模型     |
| 分块大小  | 500 字符                 | 每块约 100-200 tokens |
| 分块重叠  | 50 字符                  | 避免边界信息丢失           |
| 检索方式  | similarity             | 余弦相似度              |
| Top-K | 5                      | 取前 5 个候选           |
| 相似度阈值 | 0.5                    | 低于此值视为不相关          |

4.4 阈值过滤机制

新增的 `rag_empty` 机制解决了"查不到时编造"的问题:

RAG 检索 → 获得 [(doc1, 0.82), (doc2, 0.64), (doc3, 0.31)]

↓

过滤 `score >= 0.5`

↓

[(doc1, 0.82), (doc2, 0.64)]

↓

判断: `rag_empty = False`, 正常使用

RAG 检索 → 获得 [(doc1, 0.32), (doc2, 0.21), (doc3, 0.11)]

↓

过滤 `score >= 0.5`

↓

空列表

↓

判断: `rag_empty = True`

→ `prompt` 注入"没有相关数据, 不要编造"

→ LLM 回复"抱歉, 我没有这门课程的信息"

## 4.5 对话记忆系统 (Conversation Memory)

新增的对话记忆系统实现了跨会话持久化记忆, 使用独立的 Milvus 集合 `conversation_memory`。

每次用户对话结束后:

`call_llm` → `store_memory` → `END`

|

▼

|                                  |  |
|----------------------------------|--|
| 存入 Milvus 集合                     |  |
| <code>conversation_memory</code> |  |
| Document:                        |  |
| "User: ... \n Bot: ..."          |  |
| metadata:                        |  |
| <code>user_id</code> : 123456    |  |
| <code>source</code> : "conv..."  |  |

用户下次发消息时:

classify\_intent → retrieve\_memory → build\_prompt → ...



|                   |
|-------------------|
| 语义检索 Milvus       |
| expr: user_id==N  |
| query: 用户当前消息     |
| threshold >= 0.45 |
| k=5               |

设计要点：

| 特性    | 说明                                             |
|-------|------------------------------------------------|
| 集合    | conversation_memory ，与课程文档 course_documents 隔离 |
| 嵌入模型  | 复用 EMBEDDING_MODEL （text-embedding-3-small ）   |
| 多用户隔离 | 每条记忆带 user_id 元数据，检索时通过 Milvus expr 过滤         |
| 相似度阈值 | 0.45（略低于课程 RAG 的 0.5，因为记忆检索可以更宽松）              |
| Top-K | 5 条，排序后反转（旧→新）注入 prompt                        |
| 降级策略  | Milvus 不可用时静默跳过，对话不受影响（只是无记忆）                  |
| 不存储情形 | 无回复内容（如视频/文档分析的回复）不存储                          |

记忆注入 prompt 示例：

Your past conversations with this student:

Relevant conversation history (from oldest to most recent):

--- (relevance: 0.78)

User: COMP7940 有什么作业?

Bot: COMP7940 的作业包括 Chatbot Project...

--- (relevance: 0.65)

User: 那个项目截止日期是什么时候?

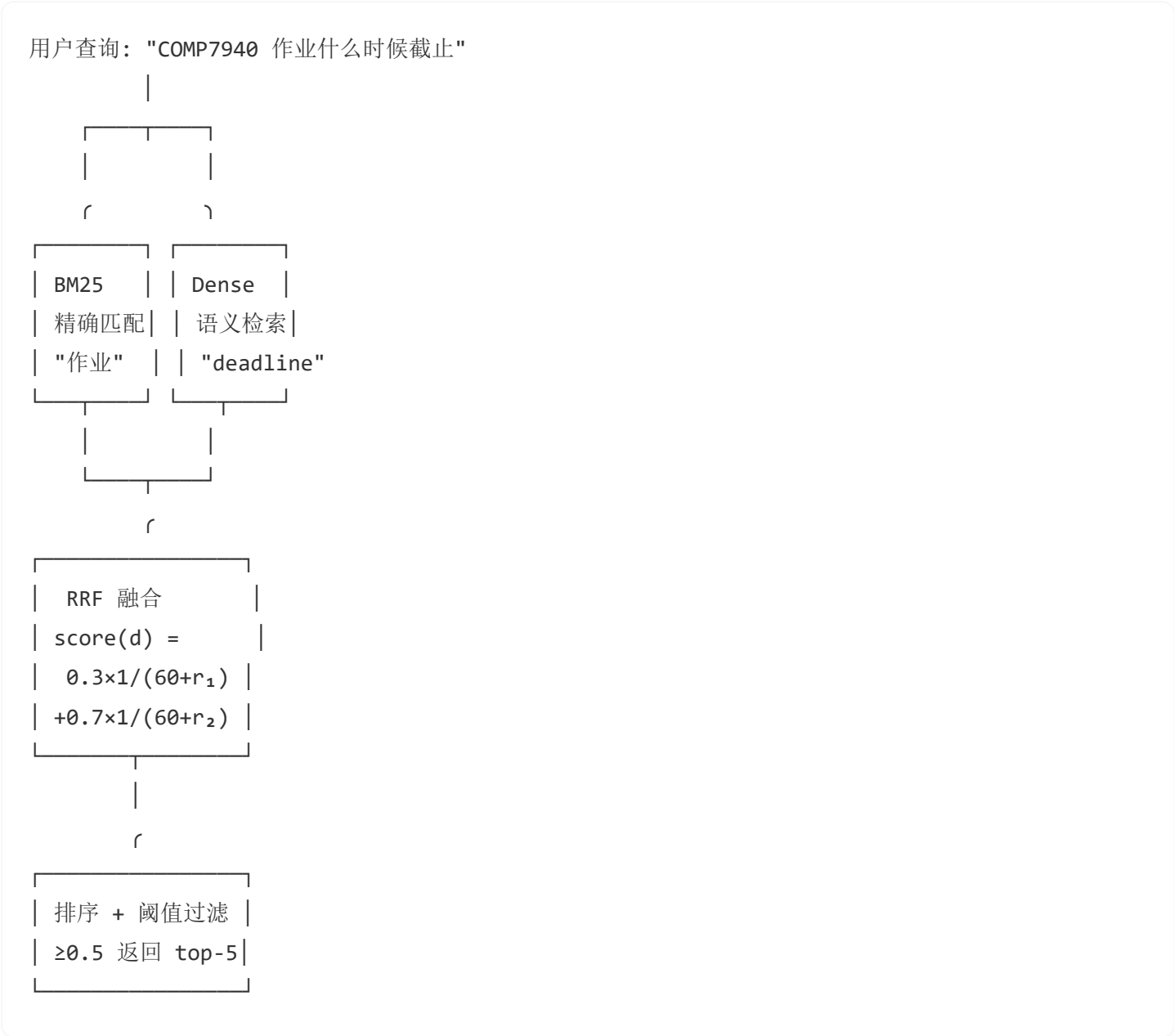
Bot: Chatbot Project 的截止日期是 2025-04-15...

(Note: The above is the student's past conversation history. Use it to maintain continuity – refer back to previously discussed topics when relevant. Do NOT mention that you are reading from a database or memory store.)

Student Question: 我想提交了，怎么交？

## 4.6 混合检索（Hybrid Search — BM25 + Dense Vector + RRF）

在纯稠密向量检索基础上，新增 **BM25 稀疏检索** 并用 **RRF（Reciprocal Rank Fusion）** 融合结果。



设计要点：

| 特性   | 说明                                            |
|------|-----------------------------------------------|
| 稀疏检索 | BM25Okapi ，从 Milvus 中加载全部文档文本构建索引，每 10 分钟自动刷新 |

| 特性   | 说明                                                                          |
|------|-----------------------------------------------------------------------------|
| 稠密检索 | 既有 OpenAIEmbeddings → Milvus cosine similarity ( k×3 候选)                    |
| 融合算法 | 加权 RRF: sparse_weight=0.3 , dense_weight=0.7                                |
| 降级策略 | BM25 不可用时 (集合为空/依赖缺失), 自动回退到纯稠密检索                                           |
| 配置项  | settings.HYBRID_DENSE_WEIGHT / HYBRID_SPARSE_WEIGHT / HYBRID_SEARCH_ENABLED |

与纯稠密对比：

| 场景                           | 纯 Dense             | Hybrid (BM25 + Dense)           |
|------------------------------|---------------------|---------------------------------|
| 精确课程代码 "COMP7940 deadline? " | 可能排名低 (代码向量不常见)     | BM25 精确命中 "COMP7940", 排名靠前      |
| 语义相近 "作业什么时候交? "             | 能召回 "assignment" 相关 | 同时命中 "作业" 关键词 + "assignment" 语义 |
| 稀有术语 "NLP transformer"       | 需 embedding 训练过该术语  | BM25 直接匹配稀有关键词                  |
| 长尾查询                         | 语义泛化可能不精准           | 关键词精确锁定 + 语义补全                  |

## 5. 数据流分析

### 5.1 文本对话流

```
User: "Explain cloud computing concepts"
├─ chatbot_agent.handle_text()
│   └─ 构建 AgentState
│       └─ await langgraph_app.ainvoke(state)
├─ [Graph] classify_intent → "general_chat"
└─ [Graph] retrieve_rag  (-- 新增: 所有对话先过 RAG)
```

```

|   └─ Milvus course_documents 集合语义检索
|   └─ 命中课程 → rag_context 注入 prompt
|   └─ 未命中 → rag_context 为空
└─ [Graph] retrieve_memory
|   └─ Milvus conversation_memory 集合语义检索 (user_id 过滤)
|   └─ 找到相关历史 → conversation_memory_context
|   └─ (第一次对话/无相关 → 空字符串)
└─ [Graph] build_prompt → "[RAG上下文][记忆上下文]...Student Question: Explain..."
└─ [Graph] call_llm → ChatGPT API → response
└─ [Graph] store_memory
|   └─ 将本轮对话嵌入向量
|   └─ 存入 Milvus conversation_memory (带 user_id 元数据)
|
└─ chatbot_agent → update.message.reply_text(response)

```

## 5.2 课程查询流（精确匹配 + RAG 回退）

User: "COMP7940 什么时候上课? "

```

|
└─ chatbot_agent.handle_text()
└─ [Graph] classify_intent → "retrieve_course", course_code="COMP7940"
|
|
└─ [Graph] retrieve_rag_node
|   └─ Milvus 语义搜索 "COMP7940"
|   └─ top-5 带分数: [(doc, 0.91), (doc, 0.87), ...]
|   └─ 过滤 score >= 0.5 → 保留全部 5 个
|   └─ rag_context = "Course Code: COMP7940..."
|
|
└─ [Graph] retrieve_memory
|   └─ Milvus 检索该用户相关历史对话
|   └─ conversation_memory_context = 历史记录
|
└─ [Graph] build_prompt
|   └─ rag_context + conversation_memory_context + user_message
|   └─ augmented_prompt = "...[RAG]...[记忆]...学生问题..."
|
└─ [Graph] call_llm → ChatGPT → final_response
└─ [Graph] store_memory → 本轮对话存入 Milvus conversation_memory

```



```
|
└─ Bot: "COMP7940 的上课时间是周一 14:30-17:15, 地点在 DLB 514..."
```

## 5.3 未知课程查询流（低于阈值）

```
User: "COMP7650 是什么课?"
|
└─ [Graph] classify_intent → "retrieve_course", course_code="COMP7650"
|
└─ [Graph] retrieve_rag_node
|   └─ Milvus 搜索 "COMP7650"
|       └─ top-5: [(COMP7940, 0.38), (COMP7930, 0.29), ...]
|           └─ 过滤 score >= 0.5 → 全部低于阈值 ❌
|               └─ rag_context = "", rag_empty = True
|
└─ [Graph] build_prompt
|   └─ rag_context 为空, rag_empty=True
|       └─ 注入指令: "没有找到相关信息, 请如实告知"
|           └─ augmented_prompt = "[安全指令]...学生问题..."
|
└─ [Graph] call_llm → ChatGPT
└─ Bot: "抱歉, 我没有找到 COMP7650 的相关信息。"
"
```

## 5.4 视频生成 workflow

涉及两次 Telegram 消息交互 + Celery 后台任务:

```
User: /video
|
└─ Bot 回复 "请发送一张图片"
    └─ user_data["waiting_for_video_image"] = True

User: [发送图片]
|
└─ handle_attachment → _route_video_image → Graph
|
└─ [Graph] receive_video_image_node
|   └─ 下载图片 → base64 编码
|       └─ Celery: analyze_image_task.apply_async(queue="ocr")
```

```
|   └─ result = task.get(timeout=30)
|
|─ Bot 回复 AI 分析结果 + 3 个动画建议
└─ user_data["waiting_for_video_prompt"] = True
```

User: "1" 或 "smooth zoom" 或 "default"

```
|
|─ handle_text → Graph
|
|─ [Graph] receive_video_prompt_node
|   └─ 解析 prompt (数字选择/自定义/default)
|       └─ Celery: generate_video_task.apply_async(queue="video")
|           └─ asyncio.create_task(monitor_video_task(...))
|
|─ Bot 回复 "视频生成已开始！"
```

[...后台监控...]

```
|
|─ position=2 → "你的视频在队列中 (位置: 2)"
|─ status=InProgress → "视频正在生成中..."
|─ task.ready() → 下载视频
└─ 发送视频文件给用户
```

## 5.5 文档分析流

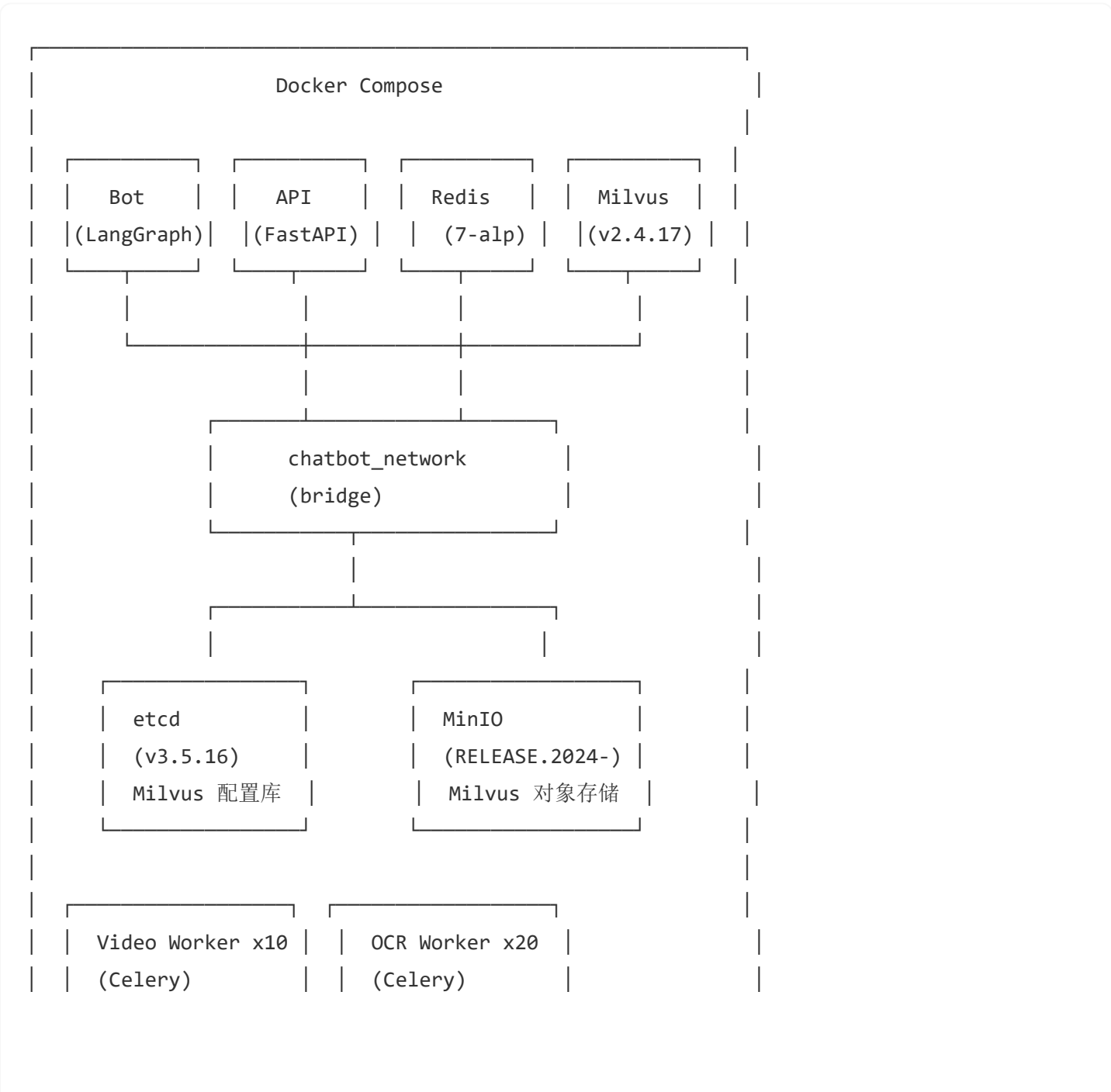
User: [发送 PDF 文件 + caption "只分析第三章"]

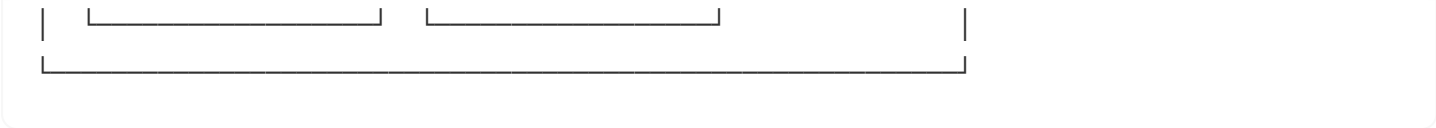
```
|
|─ handle_attachment → _route_document_analysis → Graph
|
|─ bot.py 捕获 caption → user_message = "[Document: 文件.pdf] 只分析第三章"
|
|─ [Graph] analyze_document_node
|   └─ 下载 PDF 到本地临时文件
|       └─ Celery: analyze_document_task.apply_async(queue="ocr")
|           └─ result = task.get(timeout=180)
|               └─ 异步 ingest_text() 将全文入库 Milvus (fire-and-forget)
|                   └─ 返回 {"rag_context": 全文内容 + 摘要, "rag_empty": False}
|
|─ [Graph] retrieve_memory (← 汇入对话 pipeline)
|   └─ Milvus 检索该用户相关对话历史
|
|─ [Graph] build_prompt
```

```
| └─ rag_context (PDF 全文) + conversation_memory_context + user_message
| └─ "Additional information:\n[Uploaded Document: 文件.pdf]\n..."
|
|─ [Graph] call_llm → ChatGPT (回答"只分析第三章"的具体问题)
|─ [Graph] store_memory → 本轮对话存入 Milvus conversation_memory
|
└─ Bot 回复针对性分析结果 (仅第三章内容, 而非全文档摘要)
```

## 6. 容器化部署

### 6.1 服务拓扑





## 6.2 资源分配

| 服务           | 镜像                        | 副本 | CPU 限制 | 内存限制 | 端口映射        |
|--------------|---------------------------|----|--------|------|-------------|
| bot          | python:3.12-slim          | 1  | 不限制    | 不限制  | -           |
| api          | python:3.12-slim          | 1  | 不限制    | 不限制  | 8000        |
| redis        | redis:7-alpine            | 1  | 不限制    | 不限制  | 6379        |
| etcd         | quay.io/coreos:v3.5.16    | 1  | 不限制    | 不限制  | -           |
| minio        | minio/minio:RELEASE.2024- | 1  | 不限制    | 不限制  | 9000, 9001  |
| milvus       | milvusdb/milvus:v2.4.17   | 1  | 不限制    | 不限制  | 19530, 9091 |
| video_worker | python:3.12-slim          | 10 | 2      | 2G   | -           |
| ocr_worker   | python:3.12-slim          | 20 | 2      | 2G   | -           |

## 6.3 网络拓扑

所有服务位于 chatbot\_network （bridge 网络），通过容器名互相访问：

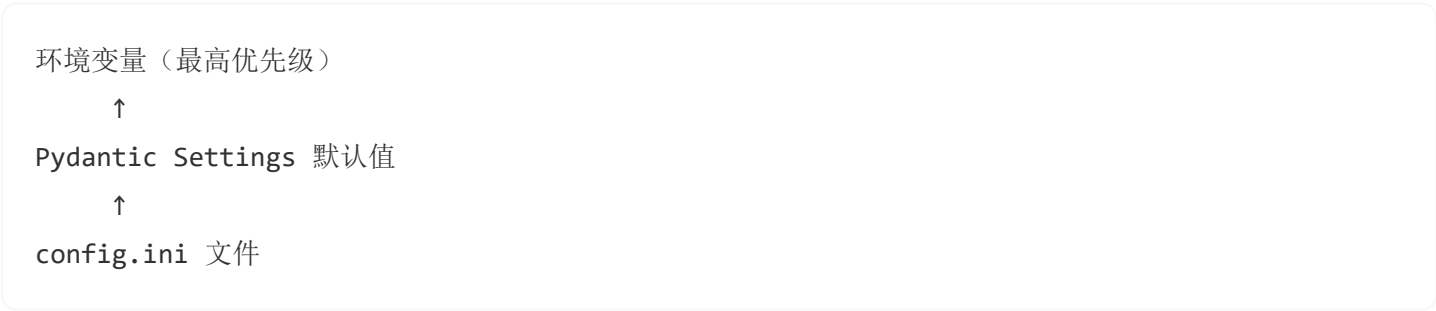
```
bot → redis:6379, milvus:19530
api → milvus:19530
video_worker → redis:6379
ocr_worker → redis:6379
milvus → etcd:2379, minio:9000
```

## 6.4 数据持久化

| 卷           | 挂载点             | 服务                | 说明            |
|-------------|-----------------|-------------------|---------------|
| redis_data  | /data           | redis             | Redis AOF 持久化 |
| etcd_data   | /etcd           | etcd              | Milvus 元数据    |
| minio_data  | /minio_data     | minio             | Milvus 向量数据文件 |
| milvus_data | /var/lib/milvus | milvus            | Milvus 内部状态   |
| bind mount  | ./config.ini    | bot, api, workers | 配置文件只读挂载      |
| bind mount  | ./logs          | bot, workers      | 日志持久化         |
| bind mount  | ./temp          | bot, workers      | 临时文件          |

## 7. 配置体系

### 7.1 配置来源优先级



### 7.2 配置项总表

| 配置项              | config.ini 段              | 环境变量                  | 默认值 |
|------------------|---------------------------|-----------------------|-----|
| Telegram Token   | [TELEGRAM] / ACCESS_TOKEN | TELEGRAM_ACCESS_TOKEN | -   |
| ChatGPT API Key  | [CHATGPT] / API_KEY       | CHATGPT_API_KEY       | -   |
| ChatGPT Base URL | [CHATGPT] / BASE_URL      | CHATGPT_BASE_URL      | -   |
| ChatGPT Model    | [CHATGPT] / MODEL         | CHATGPT_MODEL         | -   |

| 配置项               | config.ini 段        | 环境变量              | 默认值                           |
|-------------------|---------------------|-------------------|-------------------------------|
| Wan-AI API Key    | [WAN_AI] / API_KEY  | WAN_AI_API_KEY    | -                             |
| Wan-AI Base URL   | [WAN_AI] / BASE_URL | WAN_AI_BASE_URL   | https://api.siliconflow.cn/v1 |
| Wan-AI Model      | [WAN_AI] / MODEL    | WAN_AI_MODEL      | Wan-AI/Wan2.2-I2V-A14B        |
| Milvus Host       | -                   | MILVUS_HOST       | milvus                        |
| Milvus Port       | -                   | MILVUS_PORT       | 19530                         |
| Milvus URI        | -                   | MILVUS_URI        | -                             |
| Milvus Token      | -                   | MILVUS_TOKEN      | -                             |
| Milvus Collection | -                   | MILVUS_COLLECTION | course_documents              |
| Embedding API Key | -                   | EMBEDDING_API_KEY | 复用 CHATGPT_API_KEY            |
| Embedding Model   | -                   | EMBEDDING_MODEL   | text-embedding-3-small        |
| Redis Host        | -                   | REDIS_HOST        | redis                         |
| Redis Port        | -                   | REDIS_PORT        | 6379                          |

### 7.3 config.ini 文件格式

```
[TELEGRAM]
ACCESS_TOKEN = 你的_Bot_Token

[CHATGPT]
API_KEY = 你的_API_Key
BASE_URL = https://你的端点/api/v0/rest
MODEL = gpt-4o-mini
API_VER = 2024-12-01-preview
```

```
[WAN_AI]
API_KEY = 你的_SiliconFlow_Key
BASE_URL = https://api.siliconflow.cn/v1
MODEL = Wan-AI/Wan2.2-I2V-A14B
```

## 8. 扩展性设计

### 8.1 Bot Agent 水平扩展

纯异步架构意味着单进程即可处理大量并发。如需更高吞吐：

```
# 启动多个 bot 实例（需使用不同的 webhook/轮询）
docker-compose up -d --scale bot=3
```

### 8.2 Worker 弹性扩缩

```
# 增加视频 worker 到 15 个（超出默认 10 个的峰值应对）
docker-compose up -d --scale video_worker=15

# 增加 OCR worker 到 30 个（超出默认 20 个，应对批量文档上传）
docker-compose up -d --scale ocr_worker=30

# 减少 worker 节省资源（非高峰时段）
docker-compose up -d --scale video_worker=5 --scale ocr_worker=10
```

默认并发容量（docker-compose.yml 默认值）：

| Worker 类型    | 副本数 | 每进程并发 | 总并发容量 | 适用场景                |
|--------------|-----|-------|-------|---------------------|
| video_worker | 10  | 2     | 20    | 1000 用户中 ~2% 同时使用视频 |
| ocr_worker   | 20  | 4     | 80    | 1000 用户中 ~8% 同时上传文档 |

## 8.3 Milvus 扩展

- **本地模式**：单节点，适合开发和中小规模
- **分布式模式**：通过配置 Milvus 集群实现水平扩展
- **Zilliz Cloud**：完全托管的云服务，设置 `MILVUS_URI` 和 `MILVUS_TOKEN` 即可

## 8.4 功能扩展点

| 功能                | 实现方式                                                                      | 影响范围                                                            |
|-------------------|---------------------------------------------------------------------------|-----------------------------------------------------------------|
| 新增意图              | <code>classify_intent_node</code> 中添加分支 + <code>workflow.py</code> 注册节点和边 | <code>graph/nodes.py</code> 、<br><code>graph/workflow.py</code> |
| 新增 RAG 数据源        | 实现新的 <code>ingest_xxx()</code> 函数，存入同一 Milvus 集合                          | <code>rag/</code> 目录                                            |
| 接入新 LLM           | 在 <code>call_llm_node</code> 中切换客户端                                       | <code>graph/nodes.py</code>                                     |
| 自定义 Milvus schema | 修改 <code>retriever.py</code> 中的 <code>_get_cached_vector_store()</code>   | <code>rag/retriever.py</code>                                   |

# 9. 高可用设计

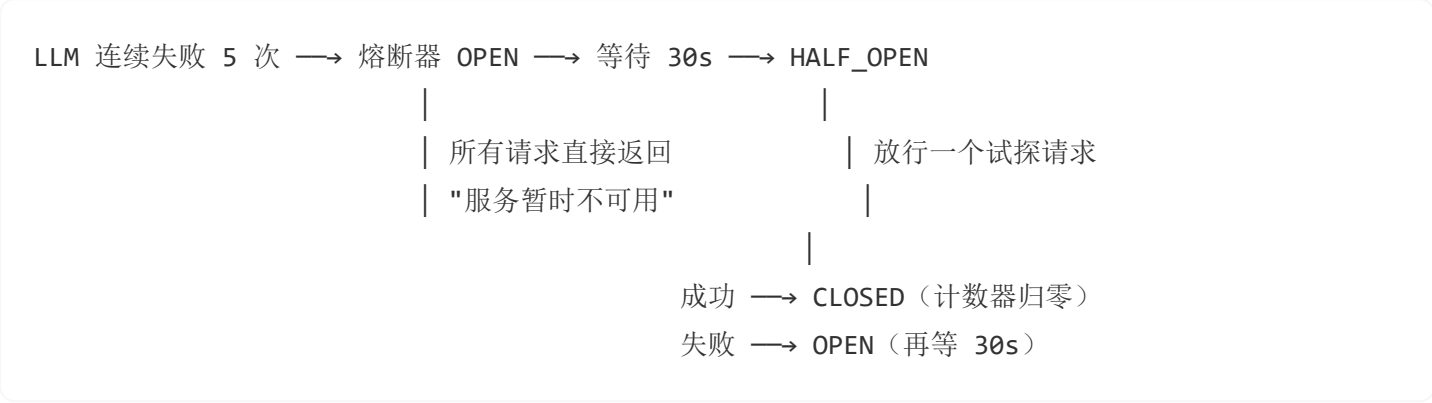
## 9.1 故障隔离矩阵

| 组件故障                      | 影响范围             | 自动恢复                     | 用户感知               |
|---------------------------|------------------|--------------------------|--------------------|
| <b>Azure OpenAI (LLM)</b> | 全部 AI 对话         | 熔断器 30s 后自愈 + 3 次重试      | "暂时不可用，请稍后再试"      |
| <b>Milvus 向量库</b>         | 课程 RAG + 对话记忆    | TTL 缓存 5min 后重建连接        | 无 RAG 上下文，对话不受阻    |
| <b>Redis</b>              | Celery 队列 + 结果存储 | Docker restart + AOF 持久化 | 视频/OCR 任务暂停，对话不受影响 |
| <b>SiliconFlow API</b>    | 视频生成             | Celery 3 次重试 + 指数退避      | 视频生成失败，可重试         |
| <b>Embedding API</b>      | RAG + 记忆向量化      | 网络超时自动重试                 | RAG 检索无结果，对话不受阻    |



| 组件故障         | 影响范围 | 自动恢复                        | 用户感知   |
|--------------|------|-----------------------------|--------|
| Telegram API | 消息发送 | python-telegram-bot<br>自动重连 | 消息延迟到达 |

9.2 熔断器机制



- 文件: ChatGPT\_HKBU.py
- 阈值: 连续 5 次失败
- 冷却时间: 30 秒
- 恢复策略: 半开状态放行单次试探，成功则关闭熔断器

9.3 速率限制与防滥用

| 防护层     | 机制                | 参数           |
|---------|-------------------|--------------|
| 每用户速率限制 | 滑动窗口计数器 (内存)      | 30 次 / 60 秒  |
| 全局并发节流  | asyncio.Semaphore | 最大 50 个并发图调用 |
| 图执行超时   | asyncio.wait_for  | 单次执行最多 30 秒  |
| 消息长度限制  | 字符数校验             | 最大 4096 字符   |
| Spam 检测 | 字符重复率分析           | >70% 重复字符拒绝  |

- 文件: chatbot\_agent.py
- 限制对象: 按 user\_id 隔离，恶意用户不影响其他用户

9.4 连接恢复策略

Milvus 连接：  
常规 → TTL 缓存（5min 过期，自动重建连接）  
失败 → `_invalidate_cache()` 立即清除缓存，下次请求重建

LLM (Azure OpenAI)：  
常规 → `httpx` 连接池（200 最大连接，50 `keepalive`）  
失败 → 3 次重试（2s→4s→8s 退避）→ 熔断器保护

## 9.5 Celery 任务可靠性

| 任务               | 重试次数 | 退避策略                         | 超时     |
|------------------|------|------------------------------|--------|
| generate_video   | 3 次  | 指数退避 2s→4s→8s（max 60s）+ 随机抖动 | 60 min |
| analyze_document | 3 次  | 指数退避 + 随机抖动                  | 60 min |
| analyze_image    | 3 次  | 指数退避 + 随机抖动                  | 60 min |

Redis AOF 持久化确保队列在容器重启后不丢失。

## 9.6 Docker 健康检查

| 服务     | 检查命令                                              | 间隔  | 超时  | 重试 | 启动等待 |
|--------|---------------------------------------------------|-----|-----|----|------|
| Redis  | <code>redis-cli ping</code>                       | 15s | 5s  | 3  | 10s  |
| Milvus | <code>curl -f http://localhost:9091/health</code> | 30s | 10s | 5  | 60s  |

Docker restart: `unless-stopped` 确保所有服务在崩溃后自动重启。